As you may have figured out, all of the variables that you can access from a controller are displayed. This list is dynamically updated as you execute code. For example, run the next line using `next`(you'll learn more about this command later in this guide).

```
(byebug) next
[5, 14] in /PathTo/project/app/controllers/articles_controller.rb
   5     # GET /articles.json
   6     def index
   7       byebug
   8       @articles = Article.find_recent
   9
=> 10       respond_to do |format|
   11         format.html # index.html.erb
   12        format.json { render json: @articles }
   13      end
   14    end
   15
(byebug)
```

And then ask again for the instance_variables:

```
(byebug) instance_variables.include? "@articles"
true
```

Now `@articles` is included in the instance variables, because the line defining it was executed.
You can also step into **irb** mode with the command `irb` (of course!). This way an irb session will be started within the context you invoked it. But be warned: this is an experimental feature.

The `var` method is the most convenient way to show variables and their values. Let's let `byebug` help us with it.

```
(byebug) help var
v[ar] cl[ass]                   show class variables of self
v[ar] const <object>            show constants of object
v[ar] g[lobal]                  show global variables
v[ar] i[nstance] <object>       show instance variables of object
v[ar] l[ocal]                   show local variables
```

This is a great way to inspect the values of the current context variables. For example, to check that we have no local variables currently defined.

```
(byebug) var local
(byebug)
```

You can also inspect for an object method this way:

```
(byebug) var instance Article.new
@_start_transaction_state = {}
@aggregation_cache = {}
@association_cache = {}
@attributes = {"id"=>nil, "created_at"=>nil, "updated_at"=>nil}
@attributes_cache = {}
@changed_attributes = nil
...
```

The commands `p` (print) and `pp` (pretty print) can be used to evaluate Ruby expressions and display the value of variables to the console.

You can use also `display` to start watching variables. This is a good way of tracking the values of a variable while the execution goes on.

```
(byebug) display @articles
1: @articles = nil
```

The variables inside the displaying list will be printed with their values after you move in the stack. To stop displaying a variable use `undisplay _n_` where *n* is the variable number (1 in the last example).